

Middleware Assignment 1 – Report

Middleware Assignment 1 – Report	1
1. Problem Summary	2
2. Project Structure	2
3. Approach and Design Decisions	2
3.1 Orchestrator (workflow_orchestrator.py)	2
Phase 1 – Task collection	3
Phase 2 – SLURM array job submission	3
Phase 3 – Dependent merge job submission	3
Logging	3
3.2 SLURM Array Job (scripts/run_primes_array.sh)	4
3.3 Merge Job (scripts/merge_primes.sh + scripts/merge_primes.py)	4
4. Efficiency Mechanisms	4
5. Execution	5
On the cluster (via SSH)	5
6. Validation	6
Test 1 – count_tot_primes.py basic correctness	6
Test 2 – count_tot_primes.py single-prime range	6
Test 3 – Orchestrator task-collection phase	6
Test 4 – End-to-end merge with overlapping ranges	7
7. Submitted Files	8
8. Limitations and Known Issues	9

1. Problem Summary

Given a project directory containing:

- An arbitrary number of `sample_*` folders inside `data/`, each holding one or more `ranges_*.txt` files with pairs of integers per line,
- A script `scripts/count_tot_primes.py` that counts primes in a given `[start, end)` range,

the goal is to produce a single `results/ALL_PRIMES.txt` containing all unique primes across every range, with a final `TOTAL: count` — while leveraging SLURM to run the computationally intensive per-range jobs in parallel on cluster nodes.

2. Project Structure

```
None
project/
├── data/
│   ├── sample_01/
│   │   ├── ranges_1.txt
│   │   └── ranges_2.txt
│   └── sample_02/
│       └── ranges_1.txt
├── results/          ← partial outputs + ALL_PRIMES.txt (generated)
├── logs/             ← ORCHEST_LOG.txt + SLURM .out/.err files
├── scripts/
│   ├── count_tot_primes.py  ← original prime-counting script (unchanged)
│   ├── run_primes_array.sh  ← SLURM array job (one task per range line)
│   ├── merge_primes.sh     ← SLURM merge job (dedup + sort)
│   └── merge_primes.py     ← Python helper called by the merge job
└── workflow_orchestrator.py ← main entry point
```

3. Approach and Design Decisions

3.1 Orchestrator (`workflow_orchestrator.py`)

The orchestrator is the single entry point. It performs three logical phases:

Phase 1 – Task collection

The orchestrator walks every `data/sample_*/ranges_*.txt` file, parses each `start end` line, and builds a flat list of *(start, end, output_path)* tuples. The output path encodes the sample name, ranges file stem, and line number to guarantee uniqueness:

```
None
results/sample_01_ranges_1_line2.txt
```

All tasks are written to `scripts/tasks.txt`, one per line, in submission order. This file acts as the index for the SLURM array.

Phase 2 – SLURM array job submission

```
None
sbatch --array=1-N ... scripts/run_primes_array.sh scripts/tasks.txt scripts/
```

A **SLURM job array** is used so that all `N` range computations run **concurrently** across available cluster nodes. The `SLURM_ARRAY_TASK_ID` environment variable (1-based) selects the correct line from `tasks.txt` inside the batch script. This avoids submitting `N` independent `sbatch` calls and lets SLURM manage scheduling as a single unit.

SLURM stdout/stderr files are written to `logs/primes_<jobid>_<taskid>.{out,err}`.

Phase 3 – Dependent merge job submission

```
None
sbatch --dependency=afterok:<array_job_id> ... scripts/merge_primes.sh results/
scripts/
```

The merge job uses a **SLURM dependency** (`afterok`) so it only starts once **every** array task has completed successfully. This removes the need for polling or sleep loops inside the orchestrator. If any array task fails, the merge job is automatically cancelled by SLURM, preventing a corrupt `ALL_PRIMES.txt`.

Logging

Every action — sample folders visited, ranges files parsed, tasks created, `sbatch` calls and their returned job IDs — is appended to `logs/ORCHEST_LOG.txt` with a timestamp.

3.2 SLURM Array Job (scripts/run_primes_array.sh)

Shell

```
LINE=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$TASKS_FILE")
START=...; END=...; OUT=...
python count_tot_primes.py "$START" "$END" > "$OUT"
```

Each task reads exactly one line from `tasks.txt` using `sed -n`. The Python interpreter path (`/soft/Miniconda3/bin/python`) matches the cluster environment declared in the original `count_tot_primes.py` shebang. Output is redirected to the task-specific file in `results/`.

3.3 Merge Job (scripts/merge_primes.sh + scripts/merge_primes.py)

The shell script is a thin SLURM wrapper; the actual logic lives in `merge_primes.py`:

1. Glob all `*.txt` files in `results/` except `ALL_PRIMES.txt` itself.
2. Parse every integer line, ignoring `TOTAL:` lines.
3. Collect values into a Python `set` — deduplication is free.
4. Sort the set and write each prime on its own line, appending `TOTAL: <count>`.

Using a Python `set` instead of shell-level `sort -u` keeps the logic portable and easy to extend.

4. Efficiency Mechanisms

Mechanism	Where used	Purpose
SLURM job array (<code>--array=1-N</code>)	<code>sbatch</code> call in orchestrator	All range computations run in parallel; a single <code>sbatch</code> manages them
SLURM dependency (<code>afterok</code>)	merge <code>sbatch</code> call	Zero-polling synchronisation; merge starts only when all tasks succeed
Python <code>set</code> deduplication	<code>merge_primes.py</code>	$O(1)$ insert; no external <code>sort/uniq</code> commands needed

Mechanism	Where used	Purpose
Unique output file naming	task collection in orchestrator	Parallel writes to results/ cannot collide

5. Execution

On the cluster (via SSH)

Connect to the cluster using the credentials provided:

```
Shell
ssh -p 443 mhedas-x@aoclspv.uab.cat
# or
ssh -p 54022 mhedas-x@aolin-login.uab.cat
```

Copy the project to your remote home directory:

```
Shell
# From your local machine:
scp -P443 -r project/ mhedas-x@aoclspv.uab.cat:.
```

Run the orchestrator:

```
Shell
# From inside the project directory:
python workflow_orchestrator.py

# Or from any location:
python workflow_orchestrator.py /path/to/project
```

Monitor jobs:

```
Shell
squeue -u mhedas-x          # list running/pending jobs
sacct -j <array_job_id>     # detailed status per array task
tail -f logs/ORCHEST_LOG.txt # live orchestrator log
```

6. Validation

All tests were run locally on macOS before cluster deployment. SLURM itself was not available locally, so the batch-script logic was exercised by calling the Python and shell components directly.

Test 1 – `count_tot_primes.py` basic correctness

Verify the script enumerates the expected primes and reports the right total:

```
Shell
$ python3 scripts/count_tot_primes.py 1 10
2
3
5
7
TOTAL: 4
```

Result: the four primes in `[1, 10)` are printed and counted correctly.

Test 2 – `count_tot_primes.py` single-prime range

Edge case: range containing exactly one prime:

```
Shell
$ python3 scripts/count_tot_primes.py 2 3
2
TOTAL: 1
```

Result: a range boundary of a single prime is handled correctly.

Test 3 – Orchestrator task-collection phase

Simulate the orchestrator's scan of all four sample folders (no SLURM required):

None

Sample folders found: 4

Total tasks collected: 14

```
[sample_01] ranges_1.txt line 1: 1 -> 50
[sample_01] ranges_1.txt line 2: 51 -> 100
[sample_01] ranges_2.txt line 1: 100 -> 200
[sample_01] ranges_2.txt line 2: 200 -> 300
[sample_02] ranges_1.txt line 1: 1 -> 100
[sample_02] ranges_1.txt line 2: 300 -> 500
[sample_03] ranges_1.txt line 1: 500 -> 600
[sample_03] ranges_1.txt line 2: 900 -> 1000
[sample_03] ranges_1.txt line 3: 1000 -> 1100
[sample_03] ranges_2.txt line 1: 1100 -> 1200
[sample_03] ranges_2.txt line 2: 1200 -> 1300
[sample_04] ranges_1.txt line 1: 2000 -> 2100
[sample_04] ranges_1.txt line 2: 2100 -> 2200
[sample_04] ranges_1.txt line 3: 2200 -> 2300
```

Result: all 4 sample folders and all 14 range lines are correctly discovered and assigned unique task numbers. On the cluster this would produce a `--array=1-14` job.

Test 4 – End-to-end merge with overlapping ranges

Six partial result files were produced by running `count_tot_primes.py` manually for ranges that intentionally overlap (1–50, 51–100, and 1–100 all share the same primes in [2, 50]):

Shell

```
python3 scripts/count_tot_primes.py 1 50 > results/s01_r1_l1.txt
python3 scripts/count_tot_primes.py 51 100 > results/s01_r1_l2.txt
python3 scripts/count_tot_primes.py 100 200 > results/s01_r2_l1.txt
python3 scripts/count_tot_primes.py 200 300 > results/s01_r2_l2.txt
python3 scripts/count_tot_primes.py 1 100 > results/s02_r1_l1.txt # overlaps
above
python3 scripts/count_tot_primes.py 300 500 > results/s02_r1_l2.txt
python3 scripts/merge_primes.py results/
```

`ALL_PRIMES.txt` (last five lines shown):

None

```
...
487
```

```
491
499
TOTAL: 95
```

Result: 95 unique primes are written despite the intentional overlap of ranges. No duplicates appear in `ALL_PRIMES.txt`. The `TOTAL:` lines from partial files are correctly ignored.

7. Submitted Files

The compressed archive contains the following structure:

```
None
project/
├─ data/
│  ├─ sample_01/
│  │  ├─ ranges_1.txt    (ranges: 1-50, 51-100)
│  │  └─ ranges_2.txt    (ranges: 100-200, 200-300)
│  ├─ sample_02/
│  │  └─ ranges_1.txt    (ranges: 1-100, 300-500)
│  ├─ sample_03/
│  │  ├─ ranges_1.txt    (ranges: 500-600, 900-1000, 1000-1100)
│  │  └─ ranges_2.txt    (ranges: 1100-1200, 1200-1300)
│  └─ sample_04/
│     └─ ranges_1.txt    (ranges: 2000-2100, 2100-2200, 2200-2300)
├─ results/              (empty; populated by SLURM jobs)
├─ logs/                 (empty; populated at runtime)
├─ scripts/
│  ├─ count_tot_primes.py
│  ├─ run_primes_array.sh
│  ├─ merge_primes.sh
│  └─ merge_primes.py
└─ workflow_orchestrator.py
```


8. Limitations and Known Issues

- **count_tot_primes.py performance:** the original script uses a trial-division loop up to `num-1`, which is $O(n^2)$. For large ranges this will be slow. The exercise treats it as a black box, so it was not modified.
- **Partial failures:** if a subset of array tasks fail (e.g., node crash), the `afterok` dependency cancels the merge. The orchestrator would need to be re-run after fixing the failing tasks. A future improvement could add a `--rerun` flag that skips already-completed output files.
- **Cluster Python path:** the shebang `/soft/Miniconda3/bin/python` is hard-coded in `count_tot_primes.py` and mirrored in the SLURM scripts. If the cluster uses a module system, add `module load python` before the Python call in the batch scripts.
- **Local testing:** the orchestrator requires `sbatch` to be available. On a local machine without SLURM, only `merge_primes.py` can be tested directly.